Empowering Machine Learning With Scalable Feature Engineering and Interpretable AutoML

Hassan Eldeeb and Radwa Elshawi

***Abstract*—Automated feature engineering (FE) has gained considerable attention in academia and industry. Nevertheless, existing systems often lack practical scalability and efficiency. This article introduces BigFeat, a scalable and interpretable framework that streamlines critical phases of the machine learning pipeline: FE, model selection, and hyperparameter tuning. BigFeat presents two execution options: as a standalone FE framework, denoted as BigFeat-FE, and as an AutoML framework, referred to as BigFeat-AutoML. BigFeat-FE optimizes input feature quality with the ultimate aim of maximizing predictive performance, based on a user-defined metric. BigFeat-FE employs a dynamic feature generation and selection mechanism that systematically creates a set of expressive features. These features not only enhance prediction performance but also prioritize interpretability. BigFeat-FE employs a meta-learning technique to warm-start the optimization process, resulting in significant overall performance gains. BigFeat-AutoML, tailored for algorithm selection and hyperparameter tuning, harnesses a random search method over the space of interpretable models. We conducted extensive experiments, and the results demonstrate that BigFeat-FE consistently outperforms state-of-the-art automated FE frameworks, such as AutoFeat and scalable automatic feature engineering (SAFE), across a wide range of datasets, achieving an average performance improvement of 8.65% compared to AutoFeat and 4.71% compared to SAFE, respectively. Additionally, BigFeat-AutoML demonstrates substantial performance improvement when compared to a tree-based pipeline optimization tool (TPOT) for automating machine learning and Autosklearn, with average improvements of 0.74% over TPOT and 2.25% over Autosklearn, respectively. Furthermore, BigFeat's scalability is affirmed through its linear complexity, and execution times, averaging 20 times faster than AutoFeat and 14 times faster than SAFE.**

***Impact Statement*—The emergence of automated scalable interpretable FE is reshaping the landscape of data science and machine learning. It automates feature creation and selection, providing interpretable features while eliminating the time-consuming burden of manual FE. By streamlining the FE process, this advancement empowers data scientists to concentrate on higher-level tasks and model development. Its scalability ensures it can handle vast and complex datasets across various industries, from healthcare to finance. Interpretability takes center stage in this innovation, enhancing model trustworthiness and facilitating regulatory compliance. The automation of different stages of the machine learning pipeline, including FE, model selection, and hyperparameter tuning, holds the promise of improving predictive modeling, decision-making, and overall efficiency in data-driven applications. This heralds a new era in data science and machine learning, where the potential for transformative advancements is on the horizon.**

***Index Terms*—Automated feature engineering, automated machine learning, feature engineering (FE), feature generation, feature selection.**

Manuscript received 7 October 2023; revised 7 February 2024 and 25 March 2024; accepted 24 April 2024. Date of publication 15 May 2024; date of current version 27 February 2025. This work was supported by the project "Increasing the knowledge intensity of Ida-Viru entrepreneurship" co-funded by the European Union. This article was recommended for publication by Associate Editor Qinghua Lu upon evaluation of the reviewers' comments. *(Corresponding author: Radwa Elshawi.)*

The authors are with the Institute of Computer Science, 51009 Tartu, Estonia (e-mail: Hassan.eldeeb@ut.ee; radwa.elshawi@ut.ee).

Digital Object Identifier 10.1109/TAI.2024.3400752

I. Introduction

THE generation and consumption of data have become integral parts of our daily lives, particularly due to the widespread availability and utilization of internet technology. This increased accessibility to data has opened up vast opportunities for machines to acquire and analyze information [1]. Machine learning (ML) has effectively harnessed this abundance of data across various application domains, risk management [2], fraud detection [3], marketing technology [4], and many others. In general, building a high-performing machine learning pipeline is a complex and time-consuming process that involves several iterative steps. These steps includes data preprocessing, feature engineering (FE), model generation and model evaluation. FE entails the application of various transformation functions, such as arithmetic and aggregate operators, to the input features, resulting in the creation of more informative features. These transformations serve to enhance the quality of the data, whether it involves scaling a feature or converting a nonlinear relationship between a feature and a target class into a linear one, which is easier for the model to learn [5].

Arguably, the key step in developing a machine learning pipeline for tabular data is FE. This can be considered comparable to various data formats such as images, text, or online video clips, where representation learning plays a critical role [5]. While traditionally, FE has been the domain of highly skilled data scientists or engineers, recent developments in automated machine learning (AutoML) [6] have provided an automated alternative. AutoML holds the potential to deliver significant productivity enhancements to data scientists, machine learning engineers, and researchers alike, primarily by alleviating the burden of repetitive tasks within machine learning pipelines. In the current landscape, a multitude of tools, encompassing both open-source [7], [8], [9] and commercially available solutions [10], [11], have emerged with the shared goal of automating these labor-intensive processes. While these

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

offer complete end-to-end ML pipelines, only a limited number of them include support for FE. The progress in automated FE has the potential to alleviate the time-intensive nature of this process for machine learning professionals and facilitate the broader utilization of machine learning techniques across various applications.

The FE process includes tasks such as feature selection, combination, and arithmetic operations. The manual approach, involving the exploration of these potential strategies by human experts, can often resemble a trial-and-error process. For instance, data experts typically start by devising a FE plan, followed by model training and subsequent evaluation of model results. Drawing from their extensive experience, data scientists then refine the feature plan and repeat this iterative process until they reach a satisfactory outcome [12], [13]. In this context, automated FE, renowned for its automation and robust exploration capabilities [14], [15], [16], holds significant promise. Automated FE [17], [18], [19], [20] is particularly appealing to industry due to its exhaustive exploration schemes and reduced labor costs.

In the following, we lay out the desired properties that an automated FE framework should satisfy.

1) *User-friendly:* A framework should be easy to use and should not require tunning a large number of hyperparameters.
2) *Interpretability:* The generated features should be easily mapped to the original features by obtaining the set of transformations applied to the original features.
3) *Scalability:* A framework should be able to handle datasets with a large number of instances and features which makes distributed computing a must. More specifically, most of the parts of the automated FE framework should be easily parallelized.
4) *Model-agnostic:* A framework should learn expressive features that improve the performance of a wide range of machine learning algorithms and should not be tightly coupled to a specific machine learning algorithm.
5) *Pipeline-enabler:* A framework should be easily extended to seamlessly automate multiple stages in the machine learning pipeline, encompassing FE, model selection, and hyperparameter tuning.

We do not claim these properties to be a complete set of desiderata; however, we believe it is a good starting point toward more efficient automated FE frameworks. With the aim of fulfilling the aforementioned desiderata comprehensively and advancing the field of AutoML toward practical applications, we introduce BigFeat.

In this article, we present BigFeat, a framework designed to streamline various aspects of the machine learning pipeline. This work is an extension of our prior work [21] BigFeat-vanilla framework, which is a scalable and interpretable automated FE framework aimed at optimizing input feature quality to maximize predictive performance according to user-defined metrics. BigFeat offers two core components tailored for specific tasks: BigFeat-FE for FE and BigFeat-AutoML for algorithm selection and hyperparameter tuning. The contributions of this article are summarized as follows.

1) BigFeat-FE leverages the dynamic feature generation techniques from BigFeat-vanilla, based on computation tree logic, to ensure linear time and space complexity in each iteration, enhancing process efficiency and scalability. Furthermore, BigFeat-FE integrates the efficient feature selection pipeline from BigFeat-vanilla, identifying stable and predictive features. For optimization efficiency, BigFeat-FE distinguishes itself by employing a meta-learning technique that significantly enhances the framework's overall performance, a feature not present in BigFeat-vanilla.
2) BigFeat-AutoML supports an end-to-end interpretable machine learning pipeline. For algorithm selection and hyperparameter tuning, BigFeat-AutoML employed a random search approach to navigate the hyperparameter space of interpretable machine learning models.
3) Through a thorough series of experiments conducted on BigFeat-FE in comparison to competing frameworks such as AutoFeat and scalable automatic feature engineering (SAFE), using standardized benchmark datasets, BigFeat outperformed the state-of-the-art results across 19 out of 20 benchmark datasets on average, demonstrating substantial advancements. Furthermore, BigFeat-AutoML demonstrates a significantly greater performance improvement when compared to a tree-based pipeline optimization tool (TPOT) for automating machine learning and Autosklearn, achieving an average performance enhancement of 0.74% compared to TPOT and 2.25% compared to Autosklearn, respectively.
4) To ensure the repeatability as one of the main goals of this work, we provide access to the source codes and the detailed results of our study.[1]

The remainder of this article is organized as follows. We first review the related work in Section II. Next, we explain the problem settings in Section III. In Section IV, we introduce the proposed framework. Section V describes the details of our experimental setup in terms of used datasets, hardware configurations, and reference frameworks. The analysis and the evaluation of the proposed framework are presented in Section VI before concluding the article in Section VIII.

II. Related Work

Deep learning (DL) techniques have achieved remarkable success on a wide range of data, including text, tabular, video, and speech [5]. DL models are known for learning expressive representations from the original input features [22]. However, learning expressive features in this way has several drawbacks in practice. First, training well-performing DL models require a large amount of data, which is not always possible in many application domains. Second, DL has mostly been successful with video, image, speech, and natural language data, whereas the general numerical data types encompass a wide variety of domains and require FE. Third, DL models require training many parameters, which is time-consuming and needs lots

[1]https://github.com/DataSystemsGroupUT/BigFeat

of computational resources. Last, the features extracted from DL models are not interpretable, making it hard for machine learning practitioners to trust models based on such features. While significant progress has been made to improve the interpretability of neural networks [23], [24], this is, in practice, still not sufficient for critical domains [25]. To avoid these shortcomings, we find it crucial to rely on interpretable automated FE techniques.

A. Heuristic Search-Based Approaches

Although FE has traditionally been a manual process, human experts have also utilized specific tools and algorithms to aid in this endeavor. For instance, exhaustive search-oriented algorithms employ recursive algorithms to systematically explore all potential transformation paths [19], [26], [27], [28], [29]. Given their significant computational complexity, Cognito has improved the original algorithm by incorporating the well-known BFS or DFS algorithms [30]. Additionally, SAFE introduces a staged approach to address industrial classification tasks, enhancing efficiency and scalability through the exploration of more effective features and an improved feature selection mechanism [31]. AutoFeat, on the other hand, employs a strategy involving local region search through an iterative downsampling process [26].

B. Learning-Based Approaches

The LFE algorithm [32] pioneers the parametrization of feature transformations, primarily tailored for classification problems. However, this method simplifies the action space by neglecting high-order feature generation actions, prompting the introduction of an algorithm which represents the FE process as a graph [33]. While this approach faces challenges related to the feature explosion problem, where excessive feature columns are added to the raw data, the notable neural feature search (NFS) [17] emerged. NFS is an RL-based learning framework that relies on an RNN-based controller to generate artificial features for each feature column individually. Catch framework [34] addresses important challenges related to system efficiency by utilizing hierarchical-policy reinforcement learning to enable collaborative FE exploration and exploitation. A separate research direction adopts an evolutionary framework that offers advantages in terms of training ease, differentiable automated feature engineering (DIFER) explores features within a relatively limited action space, making it less practical [20].

C. Meta learning-Based Approaches

Several automated FE frameworks employ meta-learning techniques. ExploreKit [35] employs meta-learning to evaluate the newly constructed features and select the most promising ones. Such technique suffers from scalability; it takes several days to process moderate-sized datasets. Learning feature engineering (LFE) framework [32] employs a meta-learning technique based on extracting meta-features from prior tasks to recommend a set of useful transformations to be applied to features without relying on model evaluation or explicit feature expansion and selection. Since the meta-features do not consider the relationship between features, it works best when using unary transformations. The efficiency and scalability of the aforementioned frameworks are still far from sufficient to be used in real-world applications. High-performing frameworks with excellent efficiency, scalability, and interpretability are in high demand.

III. Problem Statement

Let $\mathcal{D}$ be a dataset with input–output pairs (X, Y), where X is the input features and Y is the target variable. Let $X_i \in X$ be a record in the input space with N features $\{X_i^1, \ldots, X_i^N\}$. Let $Y_i \in Y$ be the corresponding output label. The training dataset consists of M instances denoted $\mathcal{D}_{\text{train}} = \{X_{\text{train}}, Y_{\text{train}}\} = \{(X_1, Y_1), \ldots, (X_M, Y_M)\}$. Similarly, validation and testing datasets are denoted $\mathcal{D}_{\text{valid}}$ and $\mathcal{D}_{\text{test}}$. In a predictive modeling task, a machine learning algorithm $\mathcal{A}$ takes $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{valid}}$ as input and produces a predictive function $\mathcal{F}: X \to Y$, which returns the predicted label as an output. Let $\mathcal{L}$ denotes the loss of the learned function $\mathcal{F}$ on $\mathcal{D}_{\text{valid}}$. The goal of the automated FE is to learn a generation function $\Psi: X \to \mathbb{X}$ to generate new expressive features representation $\mathbb{X}$ from the original input features X by applying a set of operators $\mathcal{O} = \{o_1, ..., o_{|\mathcal{O}|}\}$ to minimize the loss $\mathcal{L}$ [29], [31]. Formally, this automated FE problem can be stated as follows:

$$\Psi^* = \arg\min_{\Psi} \mathcal{L}(\mathcal{F}(\Psi(X_{\text{test}})), Y_{\text{test}}) \tag{1}$$

$$F^* = \arg\min_{F} \mathcal{L}(\mathcal{F}(\Psi^*(X_{\text{test}})), Y_{\text{test}}) \tag{2}$$

in which the predictive function $\mathcal{F}$ can be obtained by $\mathcal{F} = \mathcal{A}(\mathcal{D}_{\text{train_new}}, \mathcal{D}_{\text{valid_new}})$. $\mathcal{D}_{\text{train_new}}$ is the generated training split, i.e., $\mathcal{D}_{\text{train_new}} = \Psi(X_{\text{train}}, Y_{\text{train}})$. Likewise, $\mathcal{D}_{\text{valid_new}}$ and $\mathcal{D}_{\text{valid_new}}$ can be generated.

We assume that each generated feature $\mathbb{X}^i$ can be obtained by applying an operation $o \in \mathcal{O}$ over one or more of the raw input features. The operation o can be a unary, binary, ternary, or (n)-ary operator. For example, a unary operator could be $\log(x)$, $\sqrt{x}$, $1/x$, x^2, x^3, $\sin(x)$, e^x. A unary operator also includes normalization, discretization, or any mathematical transformation of a single feature. Binary operators combine two features to generate a new one. For numeric features, examples of binary operators include arithmetic sum $(+)$, difference $(-)$, multiplication $(\times)$, and division $(\div)$. Another example of binary operators includes logical operators acting on two boolean features such as conjunction $(\wedge)$ and disjunction $(\vee)$. Grouping operators are examples of binary operators such as GroupByThenMax, GroupByThenMin, GroupByThenAvg, and GroupByThenStdev. Ternary operators are applied to three features to generate a new one. A common ternary operator is a conditional operator. For example, as in many programming languages, the condition a? b:c, which is read as if the value of feature a is true, then the value of feature b is obtained; otherwise, the value of the feature c is obtained. For n-ary operators, many operators accept n features such as MAX, MIN, Avg, Mean, etc. It is worth mentioning that many operators are applicable in some specific domains. These operators, such as LAG operators in time series and genetic operators in biology, are called domain-specific operators. Subsequently, an automated FE framework

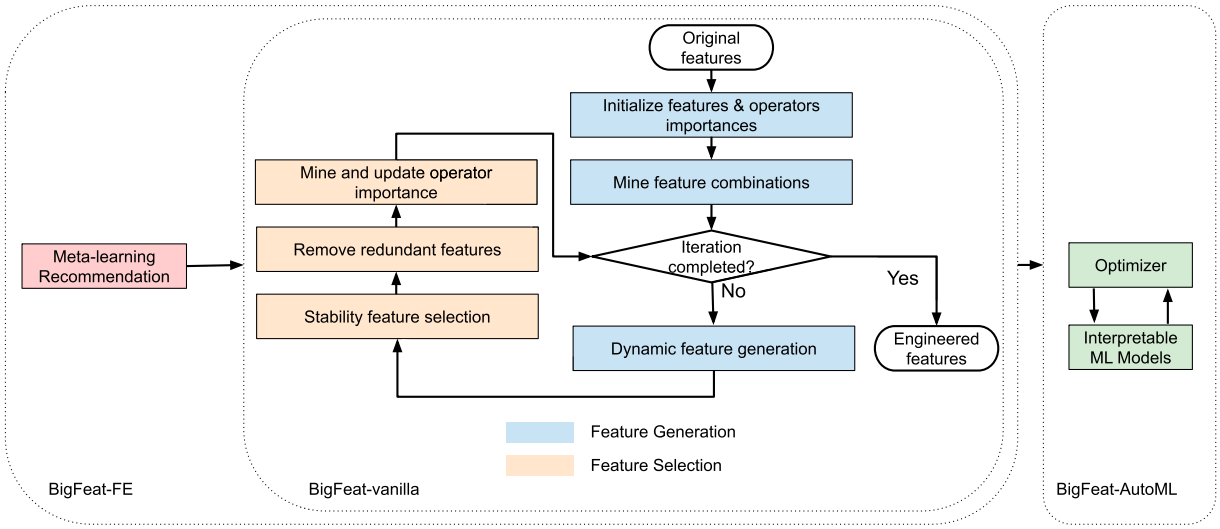

Fig. 1. Flowchart of BigFeat.

should not limit operators, and allow new operators to be added easily.

The problem of combined algorithm selection and hyperparameter tuning (CASH) for supervised task can be formally stated as follows:

Let $A = \{A^{(1)}, \ldots, A^{(R)}\}$ be a set of algorithms, and let the hyperparameters of each algorithm $A^{(j)}$ have domain $\Lambda^{(j)}$. Further, let $\mathcal{D}_{\text{train_new}} = \Psi(\mathcal{D}_{train})$ be the training dataset after undergoing FE which is split into K cross-validation folds $\{\mathcal{D}^{(1)}_{\text{valid}}, \ldots, \mathcal{D}^{(K)}_{\text{valid}}\}$ and $\{\mathcal{D}^{(1)}_{\text{train}}, \ldots, \mathcal{D}^{(K)}_{\text{train}}\}$ such that $\mathcal{D}^{(i)}_{\text{train}} = \mathcal{D}_{\text{train_new}}\ \mathcal{D}^{(i)}_{\text{valid}}$ for $i = 1, \ldots, K$, where Ψ represents the generation function learned during the FE stage and $\mathcal{D}_{\text{train}}$ is the training dataset. Finally, let $\mathcal{L}(A^{(j)}_{\lambda}, \mathcal{D}^{(i)}_{\text{train}}, \mathcal{D}^{(i)}_{\text{valid}})$ denote the loss that algorithm $A^{(j)}$ achieves on $D^{(i)}_{\text{valid}}$ when trained on $\mathcal{D}^{(i)}_{\text{train}}$ with hyperparameters λ. Then, the CASH optimization problem is to find the joint algorithm and hyperparameter setting that minimizes this loss

$$A^*, \lambda^* \in \underset{A^j \in A, \lambda \in \Lambda^{(j)}}{\arg\min} \ \frac{1}{K} \sum_{i=1}^{K} \mathcal{L}(A^{(j)}_{\lambda}, \mathcal{D}^{(i)}_{\text{train}}, D^{(i)}_{\text{valid}})$$

IV. Proposed Method

A. Overview of BigFeat

Algorithm 1 and Fig. 1 give an overview of BigFeat. BigFeat offers two distinct FE execution options: BigFeat-FE and BigFeat-vanilla. Both options share similar feature generation and selection phases, with BigFeat-FE incorporating meta-learning to optimize its performance, while BigFeat-vanilla does not. Furthermore, BigFeat-AutoML automates algorithm selection and hyperparameter tuning through a random search across an interpretable machine learning model search space.

As detailed in Algorithm 1 and Fig. 1, BigFeat-vanilla is structured as an iterative process, with each iteration encompassing two distinct phases: feature generation and feature selection. The specific number of iterations is contingent upon the available resources, such as the allocated time budget and computational capacity. The exhaustive generation of all possible features, considering the current set of base features and all operators, results in a substantial proliferation of the feature space. This, in turn, renders the exhaustive search-oriented approach nonscalable. To mitigate this challenge, we employ

Algorithm 1: Pseudo-code of BigFeat

Inputs : $\mathcal{D}_{train}$; Operators $\mathcal{O}$; Batch size of generated features K; Number of original features N; Number of iterations $Iterations$.
Output: feature generation function (Ψ)

```
I^o ← Initialize the operator importance score;
I^f ← Initialize the feature importance score;
Mine feature combinations in parallel and compute matrix C (Equation 6);
iterator ← 0;
while iterator < Iterations do
    k ← 0; X_train ← φ;
    height ← samplefrom[1 : 4];
    while k < (K × N) in parallel do
        X_train ← X_train ∪ Generate a candidate feature X^k_train with height = height (See Algorithm 3);
        k ← k + 1;
    end
    X*_train ← Select the most important features from X_train ∪ X_train and remove redundant features (See section IV-C);
    Update the operators importance scores I^o;
    iterator ← iterator + 1;
end
The feature generation function (Ψ) is obtained from the selected features (X*_train);
A*, λ_* ← find_best_model(A, Ψ(D_train))
return Ψ, A*, λ*;
```

a combination of tree-based models to mine the relationships within the set of base features, denoted as X_{train}. This is achieved by calculating an importance score for each feature, based on the information gain (IG) derived from tree-based models. Features with higher importance scores are afforded a greater likelihood of participation in the generation phase. Subsequently, we apply a predefined set of operators to the original features to generate the feature set $\mathbb{X}_{\text{train}}$. To prevent the uniform application of these operators across all new feature combinations, we assign an importance score to each operator. This score reflects the frequency with which operators have contributed to the generation of expressive features in previous iterations, with operators showing higher contribution receiving higher scores. Finally, we combine the generated feature set $\mathbb{X}_{\text{train}}$ with the base feature set X_{train} to create a comprehensive feature set.

Given that the combined feature set $\mathbb{X}_{\text{train}} \cup X_{\text{train}}$ remains large, we implement an efficient selection method to retain only a subset of N features during each iteration. The underlying principle involves identifying the most informative, stable, and nonredundant set of N features. We initially leverage the importance score to pinpoint the most informative features that exert a substantial influence on the target label. Subsequently, we employ the Pearson correlation coefficient to eliminate redundant features that may convey overlapping information. Finally, we update the importance scores for the operators based on their frequency of appearance within the selected features.

BigFeat-vanilla is thoroughly discussed in Sections IV-B and IV-C, with main focus on feature generation and selection. The meta-learning stage of BigFeat-FE is covered in Section IV-D, while Section IV-E introduces BigFeat-AutoML.

B. Feature Generation

The feature generation stage aims to produce a set of informative features by leveraging the existing base feature set, denoted as X_{train}. The aim is to retain the most promising N features during each iteration, ensuring that they are not overlooked.

The size of the search space (S) of original features X_{train} and operators $\mathcal{O}$ is

$$| S |= \sum_{i=1}^{N} \binom{N}{i} \times | \mathcal{O}_i | \tag{3}$$

where $\binom{N}{i}$ is the number of ways to select i features from N features, and $| \mathcal{O}_i |$ is the number of i-ary operators. For example, for i equals 2, the size of this search space portion equals to the number of ways to select two features out of N features $\binom{N}{2}$ multiplied by the number of binary operators we have in our portfolio ($| \mathcal{O}_2 |$). $| S |$ is the summation overall $i < N$. It is worth mentioning that, for simplicity, we counted $\mathcal{O}_i(X^i, X^j)$ and $\mathcal{O}_i(X^j, X^i)$ as one while calculating $| S |$.

In general, the feature generation stage consists of the following steps.

1) Initializing the Importance Score of Features and Operators: We calculate an importance score for each base feature and each operator in our portfolio. To initialize the importance score for each feature, we train a tree-based model on D_{train} to score the importance of the base features X_{train} based on the IG value. Such scores are then normalized to construct the feature importance vector I^f

$$I^f = \begin{pmatrix} I_1^f & I_2^f & \dots & I_N^f \end{pmatrix} \tag{4}$$

where I_i^f is the importance of feature X^i.

To initialize the importance scores for operators, a uniform distribution is employed, as there is no inherent preference for any specific operator initially. The operator importance vector, denoted as I^o, is initialized as follows:

$$I^o = \begin{pmatrix} I_1^o & I_2^o & \dots & I_{|\mathcal{O}|}^o \end{pmatrix} = \begin{pmatrix} 1 & 1 & \dots & 1 \end{pmatrix}. \tag{5}$$

2) Mine Feature Combinations: As mentioned previously, the vastness of the search space necessitates our focus on generating expressive features. This entails affording greater opportunities to features that are hypothetically expressive, all while ensuring that potential features with the capacity to become expressive in future iterations are not overlooked. Consequently, our automated FE framework relies on two fundamental assumptions: 1) new features generated from split features of a tree-based model have the potential to be more efficient than features derived from nonsplit features; and 2) new features generated from split features along the same path within a tree-based model are likely to be more expressive than those originating from split features along different paths. In accordance with these assumptions, we assign higher probabilities for selecting split features from the same paths together. We maintain a matrix denoted as C, which records the frequency of occurrences for each pair of split features sharing the same path (as outlined in Algorithm 1, line 3)

$$C = \begin{pmatrix} C_{1,1} & C_{1,2} & \cdots & C_{1,N} \\ C_{2,1} & C_{2,2} & \cdots & C_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ C_{N,1} & C_{N,2} & \cdots & C_{N,N} \end{pmatrix} \tag{6}$$

where $C_{i,j}$ is the number of occurrences of X^i and X^j together in the same path in a tree-based trained model. $C_{i,j}$ is initialized to 1 and incremented by 1 for each path where both X^i and X^j appear together.

3) Dynamic Feature Generation: To tackle the scalability challenge, BigFeat-FE adopts computation trees for the dynamic generation of KN features per iteration, with K significantly smaller than N. These computation trees are rooted structures where each internal node denotes a distinct computational state, while leaf nodes represent base features, and edges signify transitions to potential computations. Examples of computation trees are illustrated in Fig. 2. The depth of a node within the computation tree corresponds to the length of the path from the root to that specific node. In contrast, the tree's height, denoted as *height*, represents the maximum distance between the root and any leaf node. The *height* is determined through random sampling, with a bias toward favoring simpler and shallower computational trees.

The construction of a computation tree follows a depth-first approach. It starts with a single root node representing an operator sampled from our portfolio, with selection probability

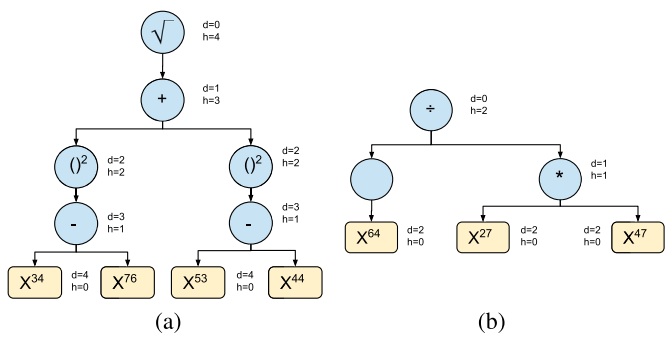

Fig. 2. Examples of computation trees to generate two features. (a) Euclidean distance between two points (X^{34}, X^{53}) and (X^{76}, X^{44}). (b) Price per meter square given the price X^{64}, length X^{27} and width X^{47}. Each internal node in each tree corresponds to an operator, and each leaf node corresponds to a base feature. We first sample the square root operator ($\sqrt{\ }$) and binary division operator ($\div$) at the root nodes of the Euclidean distance tree and price per meter tree, respectively. Then we grow all nodes by sampling operators as long as *height* of the left subtree is more than 0, and then we sample features at *height* = 0. For the Euclidean distance tree and price per meter tree, we sample two features X^{34} and X^{76} and one feature X^{64}, respectively. Before backtracking, all other branches are grown similarly. Finally, the Euclidean distance feature and price per meter are obtained by evaluating the computation tree and applying the operator at the root to values obtained by recursively evaluating the left and right subtrees. The Euclidean distance feature $= \mathbb{X}^i = \sqrt{(X^{34} - X^{76})^2 - (X^{53} - X^{44})^2}$ and the price per meter $= X^{64} \div (X^{27} \times X^{47})$.

Algorithm 2: Dynamic Feature Generation

Inputs : $\mathcal{D}_{train}$; Operators $\mathcal{O}$; Operators importance vector I^o; Feature Importance vector I^f; computational tree height $height$; Selected Features = ϕ.
Output: Generated Feature $\mathbb{X}^i$.

```
if height = 0 then
    return a sampled feature from X_train with a
      probability of selection proportional to its
      importance score in I^f;
else
    o ← sample a new (n)-ary operator from O with
      probability of selection proportional to its
      importance score in I^o;
    for each operand of (n)-ary operator do
        New Feature ← Dynamic Feature Generation(...,
          height − 1, Selected Features, ...);
        Selected Features ← Selected Features ∪ New
          Feature;
    end
    X^i = the result of applying o on Selected Features;
    return X^i;
end
```

based on its importance score in I^o. The tree is then expanded by growing one branch until reaching the maximum depth. Operators are sampled for nodes at *height* > 0, and features are sampled for *height* = 0, with probabilities based on their importance scores in I^o and I^f, respectively. Similar expansions occur for other branches before backtracking. Nodes at the preceding level of the tree are used to determine the number of nodes at the subsequent level. For example, if a node at the current level of the computation tree corresponds to an n-ary operator, it will have n child nodes in the subsequent level. Finally, we modify the feature importance vector I^f to assign greater probabilities to features that co-occur along the same path within the tree-based model, which was originally trained during the initialization phase. To achieve this, we update the I^f vector by multiplying it with $C_{i,:}$, where i is the index of the currently sampled feature. The pseudo-code outlining the algorithm for the dynamic feature generation phase is provided in Algorithm 3.

Algorithm 3: Offline Phase of BigFeat-FE

Inputs : $\mathcal{D}_{train}$; Operators $\mathcal{O}$.
Output: Operators models $Models$.

```
Catalog ← {};
for each dataset in Datasets do
    BigFeat-v(dataset, ...);
    best I^o ← final I^o;
    I^m ← extract_metafeatures(dataset);
    Catalog ← Catalog ∪ (I^m, best I^o);
end
Models ← {};
for each o in O do
    Catalog^ ← drop_other_operators(Catalog, o);
    model ← TPOT(Catalog^, I^o);
    Models ← Models ∪ (o, model);
end
return Models;
```

C. Feature Selection

Following each iteration, our process yields an additional set of KN newly generated features alongside the existing base features. However, these features may not all carry equal significance. To efficiently pinpoint the most representative N features, we employ a three-step feature selection technique. Initially, we sort features based on their importance, as measured by their stability and predictive performance. Subsequently, we focus on selecting the features that are not only important but also nonredundant. Finally, our approach involves updating the operator importance vector reflecting the operators' contributions to the generation of important features in previous iterations.

1) Stability Feature Selection: The optimal method for selecting N features out of a pool of $(K+1)N$ features involves trying all possible combinations, yet this approach is computationally intensive. Conversely, selecting features individually without considering their interdependencies may lead to suboptimal choices. To balance between performance and efficiency, we adopt a stability selection mechanism as an approximation to the optimal technique [36], [37]. More specifically, we train

α tree-based models, with each model being trained on a subset of the total number of features and instances. These subsets are sampled with replacement, encompassing both features and instances. Subsequently, features' importance determined by the IG from the α tree-based models, is normalized, aggregated, and sorted in descending order. This sorting process enables the selection of the most important and nonredundant N features.

2) Remove Redundant Features: We compute the Pearson correlation [38] between every pair of candidate features within $\mathbb{X}train \cup Xtrain$. The magnitude of the correlation coefficient indicates the strength of the correlation between these features. We set a threshold for the Pearson correlation, denoted as η. When the correlation coefficient between any pair of features within $\mathbb{X}train \cup Xtrain$ surpasses the η threshold, we retain the features with the higher importance score and discard the others. Finally, we select the most important and uncorrelated set of N features, which are then stored within $\mathbb{X}^*_{\text{train}}$.

3) Mine and Update Operator Importance: Following each iteration, we update the operators' importance scores based on their contributions to the generation of features within $\mathbb{X}train^*$. The importance score of each operator is increased in accordance with the count of times the operator contributed to the creation of generated features in $\mathbb{X}train^*$. More specifically, the operator importance vector I^o at iteration c is updated as follows:

$$I^o \leftarrow ((c-1)I^o + \overline{I^o})/c \tag{7}$$

where, $\overline{I^o}$ is the newly calculated importance vector I^o such that all operators appeared in forming generated features in $\mathbb{X}^*_{\text{train}}$ are incremented by their count of occurrences and eventually normalized.

D. Meta-Learning for Optimizing Operator Importance Vector in BigFeat-FE

BigFeat-FE employs meta-learning step to warmstart the operator importance vector. We employ meta-learning to identify instantiations of operator importance vectors that are more likely to play a significant role in generating important features for new datasets. In this context, our approach involves gathering performance data and extracting a set of efficiently computable meta-features from a substantial pool of diverse datasets. These meta-features encapsulate key dataset characteristics. The concept of warmstarting optimization through meta-learning has previously demonstrated success in prior works [39], [40], [41].

1) Meta-Features Extraction: Three primary categories of metafeatures have been extracted for dataset characterization [42].

1) *Statistical and Information-Theoretic Metafeatures:* These metafeatures are derived from statistical and information-theoretic properties of the dataset. They includes features such as the number of classes, number of features, the ratio of examples to features, the degree of correlation between features and the target concept, and the average class entropy. This approach has been widely utilized in literature, and has consistently yielded positive results [41], [43], [44].
2) *Model-Based Metafeatures:* This approach involves creating a decision tree from the dataset and capturing properties of the tree, such as the number of nodes per feature, maximum tree depth, tree shape, and tree imbalance. These properties collectively form a set of metafeatures [45], [46].
3) *Landmarkers:* Another approach involves harnessing insights gained from the performance of a selection of simple and fast learners, each with distinct learning mechanisms. The accuracy of these reference models, often referred to as "landmarkers," serves to characterize a dataset and pinpoint specific areas where each type of learner excels as an expert [47], [48].

To characterize datasets, we implemented a total of 104 meta-features. All meta-features are listed in Table III.

2) Selecting Good Instantiations: The meta-learning phase in our approach complements the optimization phase. Our meta-learning works as follows: In an offline phase, for each dataset in our knowledge-base consisting of 176 real and synthetic datasets, we evaluate a set of meta-features (described above) and use BigFeat-Vanilla for five iterations to report and store an instantiation of operator importance vector. For each operator, we train a meta-model on the meta-features and the importance score associated with that operator. This meta-model predicts the importance score of that specific operator when applied to a given dataset. In this training process, we employ TPOT, a powerful automated machine learning framework, to identify the best-performing machine learning model for the given operator. TPOT is run with a time budget of 10 min, optimizing for the F1 score as the primary evaluation metric. Given a new dataset D, we compute its meta-features and get the importance scores for all operators from all meta-models. These importance scores serves as an instantiation for the operator importance vector for dataset D.

E. BigFeat-AutoML

BigFeat-AutoML is an AutoML framework designed to facilitate model selection and hyperparameter tuning for classification tasks. BigFeat-AutoML is implemented on top of `Scikit-Learn` [49], a popular Python ML package, and uses random search for algorithm selection and hyperparameter tuning. To deliver a comprehensive end-to-end machine learning pipeline, BigFeat-AutoML harnesses a diverse array of interpretable models, including Random Forest, Logistic Regression, Decision Tree, and Gradient Boosting. To efficiently explore the space of possible classification pipelines, we should be able to enumerate the sets of hyperparameters which describe the recommended classification algorithm. Table I lists the supported classification algorithms along with the set of main hyperparameters.

F. Time Complexity Analysis

In this section, we conduct a time complexity analysis of BigFeat-FE, drawing comparisons with the complexity of

TABLE I
SEARCH SPACE OF BIGFEAT-AUTOML

Classifier	Hyper-parameter	Range
rf	n_estimator Max_depth Class_weight	[10, 50, 100, 250] [None, 5, 10, 20] [None, {0:1,1:5}, {0:1,1:10}, {0:1,1:25}]
lr	C Penalty Class_weight	[10^{-2}, 10^{-1}, 10^{0}, 10^{1}, 10^{2}] ['l1', 'l2'] [None, {0:1,1:5}, {0:1,1:10}, {0:1,1:25}]
dt	Max_depth Min_samples_split Class_weight	[5,10,25,None] [2,5,10] [None, {0:1,1:5}, {0:1,1:10}, {0:1,1:25}]
gb	n_estimators Max_depth	[10, 50, 100, 250] [5, 10, 20]

AutoFeat and SAFE. The complexity analysis is based on the worst-case scenario considering the default parameters of each framework.

1) BigFeat-FE: For simplicity and generality, we assume that only binary operators are used, and Random Forest is selected for features' importance and combinations. The time complexity of each component in the feature generation and feature selection phases of BigFeat-FE is as follows.

a) The primary element of the feature initialization phase involves the training of a Random Forest classifier. This training process carries a computational complexity of $O(N_1 N_2 D_1 M)$, where N_1, N_2, and D_1 represent the number of trees, the number of features in each tree, and the tree depth, respectively.
b) The initialization of operator importance using the meta learning technique incurs a computational complexity of $O(F_1 N_2 D_1 M)$, where F_1 denotes the count of extracted meta-features. Notably, the most computationally demanding meta-features are landmarking-based features. More specifically, the training of a Decision Tree model.
c) The dynamic feature generation process exhibits a complexity of $O(I_1 K N 2^{D_2} C)$, where I_1, C, and D_2 denote the iteration number, the cost of applying an operator on M instances of a given feature, and the maximum depth of generated features, respectively.
d) The stability-based feature selection operation carries a complexity of $O(\alpha N_1 N_2 D_1 M)$. The assessment of each operator's contribution to the generation of important features, prior to updating the operators' importance scores (I^o), is performed with a complexity of $O(O_2 N)$, where O_2 represents the number of operators in our portfolio.
e) To eliminate redundant features, we iterate through the best features to check for high correlations with their base features in the computational tree. This operation has a complexity of $O(O_1^{D_2} N)$, where O_1 denotes the operator category.

α, K, D_1, D_2, F_1, O_1, and O_2 are constants. The iteration number I_1 and the number of trees N_1 are user-choice parameters recommended to be 7 and 100, respectively. Hence I_1 and 2^{D_2} are considered constants. C is a function of M and N_2 depends on N.

$$\begin{aligned} O(\text{BigFeat-vanilla}) &= (\alpha+2)O(N_1 N_2 D_1 M) \\ &\quad + O(I_1 K N 2^{D_2} C) + O(O_2 N) \\ &\quad + O(O_1^{D_2} N) \\ &= O(N_2 M) + O(NM) + (2)O(N) \\ &= O(NM) \end{aligned} \tag{8}$$

$$\begin{aligned} O(\text{BigFeat-meta}) &= (\alpha+2)O(N_1 N_2 D_1 M) \\ &\quad + O(F_1 N_2 D_1 M) + O(I_1 K N 2^{D_2} C) \\ &\quad + O(O_2 N) + O(O_1^{D_2} N) \\ &= (2)O(N_2 M) + O(NM) + (2)O(N) \\ &= O(NM). \end{aligned} \tag{9}$$

It's important to highlight that the analysis provided earlier does not account for the parallelization capabilities inherent in BigFeat-FE algorithms. These parallelization techniques contribute significantly to enhancing performance compared to purely sequential approaches. Despite having linear complexity, the parallel nature of BigFeat-FE allows for efficient utilization of computational resources, leading to improved processing speed and scalability.

2) AutoFeat [26]: If AutoFeat is to run for k iteration, the time/space complexity of feature generation phase is $O(O_3 M N^{2^k})$, where O_3 is the number of operators. During the selection phase, Autofeat uses logistic regression with complexity of $O(O_3 M N^{2^k})$

$$\begin{aligned} O(\text{AutoFeat}) &= O(O_3 M N^{2^k}) + O(O_3 M N^{2^k}) \\ &= O(M N^{2^k}) + O(M N^{2^k}) \\ &= O(M N^{2^k}). \end{aligned} \tag{10}$$

3) SAFE [31]: During the generation step, SAFE employs an XGBoost classifier to identify informative feature combinations, sorts them, and utilizes the most informative ones for generating new features. However, in the subsequent selection step, SAFE introduces a more intricate process to identify redundant features by evaluating the correlation between each pair of features following the generation phase. Consequently, its time and space complexity can be characterized as follows:

$$O(\text{SAFE}) = O(NM^2). \tag{11}$$

The analysis shows that BigFeat-FE has linear complexity compared to double exponential complexity of AutoFeat and quadratic time/space complexity of SAFE.

V. EXPERIMENTAL SETUP

To ensure the robustness of our approach, we relied on two third-party benchmarks, [50] and [51], to prevent any overfitting to specific characteristics. These benchmark suite comprises a diverse range of datasets, featuring varying numbers

TABLE II
INFORMATION OF THE BENCHMARK DATASETS

Dataset	Instances	Features	Labels
Madelon	2600	500	2
Gina	3153	970	2
Arcene	200	1000	2
Nomao	34 465	118	2
Kc1	2109	21	2
Eeg-eye	14 980	14	2
Credit-g	1000	20	2
Sonar	208	60	2
Shuttle	58 000	9	7
Blood	784	4	2
Amazon_employee	32 769	9	2
Australian	690	14	2
Christine	5418	1636	2
GCM	190	16 063	14
Micro-mass	571	1300	20
Numerai28	96 320	21	2
Phoneme	5404	5	2
Isolet	7797	618	26
Steel-plates-fault	1941	28	7
Texture	5500	41	11

of instances and features. The datasets' sizes span from 200 to 96 320 instances, while the feature counts range from 4 to 16 063, as detailed in Table II. We made sure that the test datasets do not intersect with the training datasets to prevent any data leakage, and we carefully selected datasets that have minimal or zero data cleaning to avoid any potential bias from the data cleaning stage as it is not the focus of our contribution. To maintain simplicity in our BigFeat-FE evaluations, we utilized a set of fundamental operators, including arithmetic operations (addition, subtraction, multiplication, and division), as well as square and absolute operations. Our experiments were conducted in a CPU environment running on Ubuntu 18.04 LTS, equipped with a 64-core Intel Processor @ 2.00GHz and 240 GB of RAM. We assessed the performance of our constructed ML pipelines using the F1-score metric. In the feature selection phase of BigFeat for FE, we utilized both random forest and LightGBM models to evaluate feature importance. To initialize the feature importance vector in BigFeat-FE, we determined the significance of each feature using a random forest model. Our framework's performance was benchmarked in two distinct scenarios. First, we compared BigFeat-FE and BigFeat-vanilla with the state-of-the-art FE frameworks, AutoFeat and SAFE. Second, we assessed BigFeat-AutoML in comparison to state-of-the-art AutoML frameworks, AutoSklearn and TPOT. For all AutoML frameworks, we set the time budget to be 10 min. The Automated FE run is considered unsuccessful if it exceeds a duration of 24 h. For both BigFeat-FE and BigFeat-vanilla, we experimentally determined fixed values for the following parameters: $\eta = 0.8$, $K = 10$, $\alpha = 5$, and Iterations $= 7$, leaving further exploration of these parameter values to future work. K is optimized to maximize feature generation capacity within memory constraints, while the iteration number controls the depth of feature exploration for computational efficiency. Parameters η and α are strategically selected to balance feature redundancy, model flexibility, and overall performance, facilitating an effective FE process. The selection of parameters K, α, and Iterations is contingent upon available computational resources, with recommendations for increases in response to enhanced processing capabilities and memory capacity. Conversely, parameter η should be reduced to prioritize feature uniqueness within the output features.

A. Reference AutoML Frameworks

`Auto-Sklearn` is an AutoML framework for building pipelines for classification and regression tasks and is implemented on top of Scikit-Learn`Scikit-Learn`,[2] a popular Python machine learning package [52]. It uses SMAC as a Bayesian optimization technique [53]. Auto-Sklearn uses meta-learning to initialize the optimization procedure. Additionally, ensemble learning is implemented by combining the best pipelines to improve the performance of the output model. One of the main advantages of Auto-Sklearn is that it comes with different execution options. In particular, the basic version, the vanilla version (`Auto-Sklearn-v`), applies SMAC for the AutoML optimization process. However, Auto-Sklearn also allows end-users to enable/disable the different execution options including the use of meta-learning (`Auto-Sklearn-m`), ensembling (`Auto-Sklearn-e`) in addition to the full version (`Auto-Sklearn`) where all options are enabled. We used the full (default) version of `Auto-Sklearn`.

`TPOT` is an AutoML framework for building classification and regression pipelines through the utilization of a genetic algorithm. Within this framework, machine learning pipelines are represented as computational graphs, where distinct branches symbolize diverse preprocessing pipelines. Subsequently, these pipelines undergo optimization via a multiobjective optimization approach, aiming to simultaneously minimize pipeline complexity and enhance performance. This optimization strategy addresses the challenge of navigating a vast search space, reducing the risk of overfitting [54].

B. Reference Automated `FE` Framework

`AutoFeat` is a python library that provides linear regression and classification models, inheriting scikit-learn style, with automated feature generation and selection capabilities. `AutoFeat` is a general-purpose library for a single table datasets. First, AutoFeat generates all the possible combinations of features by applying predefined nonlinear operators (e.g., $\log(x), \sqrt{x}, 1/x, x^2, x^3, \sin(x), e^x$) on the input features. Next, these generated features are combined using the four main mathematical operators ($+, -, *$, and $/$). Finally, these features are normalized, and a linear model is trained to select the most important features.

`SAFE` is a two-stage method designed for efficient automatic FE. In the feature generation stage, SAFE prioritizes the identification of original feature pairs likely to yield more effective new features, enhancing efficiency. During the feature selection stage, SAFE introduces a comprehensive pipeline that considers important and nonredundant features, with feature importance assessed through a tree-based model.

[2]https://scikit-learn.org/

VI. Results

A. Predictive Performance of BigFeat for FE

To evaluate the performance of BigFeat-vanilla and BigFeat-FE, we assess the performance of both the original features and those generated by AutoFeat, SAFE, BigFeat-vanilla, and BigFeat-FE. This evaluation is conducted across a spectrum of eight state-of-the-art classification algorithms, including AdaBoost (AB), Decision Tree (DT), Extremely Randomized Trees (ET), k-Nearest Neighbors (kNN), Logistic Regression (LR), Multilayered Perceptron (MLP), Random Forest (RF), and Gradient Boosting (GBoost), as detailed in Table III. All algorithms are employed with their default hyperparameter settings, as per the scikit-learn library [52]. The reported performance is based on the mean value from a total of 10 independent runs. For each classifier, we conducted five distinct configurations, including the original raw features (ORG), engineered features derived from AutoFeat (AF), engineered features derived from SAFE (SF), engineered features derived from BigFeat-vanilla (BF-v), and engineered features obtained through the BigFeat-FE (BF-FE). The results demonstrate the superior performance of BigFeat-FE compared to the most competitive baseline, SAFE, across 19 out of 20 datasets, with an average improvement of 4.71%. Moreover, the results reveal that BigFeat-FE surpasses both the ORG and AF configurations across the majority of the datasets, achieving notable average improvements of 5.66% and 8.65% over the ORG and AF configurations, respectively, as detailed in Table III. Specifically, for multiclass datasets, BigFeat-FE has consistently demonstrated comparable performance. It consistently outperformed both SAFE and other configurations in managing the heightened complexity and diversity of classes across all six datasets, underscoring its robustness and adaptability in more challenging classification scenarios, particularly relevant for AutoML systems [55]. The average improvement on F1-Score is 3.92%, 9.38%, and 5.30% over SF, AF, and ORG, respectively. This enhancement in multiclass datasets performance further highlights BigFeat-FE's capability to handle diverse types of classification problems effectively. These findings underscore the effectiveness of BigFeat-FE in enhancing classification outcomes.

B. Feature Generation

To assess the effectiveness of the dynamic feature generation technique integrated into the various execution options within the BigFeat framework, we conducted a performance evaluation. Specifically, we compared the performance of features generated by BigFeat-vanilla to those generated randomly, applying the same selection technique as in BigFeat-vanilla, which we refer to as RFG. The performance of different classification algorithms based on BigFeat-vanilla and RFG features is presented in Table IV. All algorithms were used with their default hyperparameters as provided by scikit-learn. The results demonstrate the superiority of features generated by BigFeat-vanilla, showcasing an average improvement of 2.19% across all classifiers and datasets compared to RFG, as shown in Table IV. This underscores the remarkable effectiveness of BigFeat's dynamic feature generation approach in enhancing classification outcomes.

C. Feature Selection

To assess the efficacy of the stability-based feature selection method employed in the diverse execution options offered by BigFeat, we conducted a comparative analysis. Specifically, we evaluated the performance of the feature subsets selected by BigFeat-vanilla against those chosen by ANOVA feature selection (referred to as AFS) while utilizing the same feature generation technique as BigFeat-vanilla across various classifiers. AFS identifies the top N features based on the ANOVA F-value between labels and features. The results of this comparison revealed that the features derived from the stability-based feature selection technique consistently outperformed those selected by AFS, exhibiting an average improvement of 1.71% across all classifiers and datasets, as illustrated in Table IV.

D. Feature Importance

To assess the significance of features generated through the various execution options offered by BigFeat, we employ the following evaluation procedure. Specifically, for BigFeat-vanilla, we closely monitor the N selected features and compare the proportion of aggregated feature importance derived from the base features to that of the generated features. More specifically, we train Random Forest and Light Gradient Boosting models on the set comprising the initial N base features, augmented with the KN generated features obtained from BigFeat-vanilla. Subsequently, we calculate feature importance scores and select the top N most crucial features. Fig. 3 illustrates the fraction of base features (depicted in orange) and generated features (indicated in blue) deemed important among the top-ranked N features. Remarkably, across all datasets, the proportion of generated features within the top-ranked N exceeds 80%, as shown in Fig. 3. This compelling outcome strongly validates that the generated features significantly enhance the overall importance and utility of the feature set.

E. Performance Improvements Over Iterations

To validate whether the performance can be further improved over iterations, for each dataset, we report the performance of the best-performing machine learning model obtained from `Auto-Sklearn` trained on the features obtained from BigFeat-vanilla over five iterations, as shown in Fig. 4. The results show that the performance is improved over the early iterations with a larger difference until stabilized after a few iterations. This is mainly because most useful combinations are investigated in the early iterations, and the effect is reflected in the predictive performance. In the later iterations, the room for improvements is getting smaller; thus, the generated feature combination has slightly impacted the performance.

F. Scalability of BigFeat-Vanilla and BigFeat-FE

BigFeat-FE demonstrates linear complexity with respect to the size of the original base features. This linear complexity also applies to BigFeat-vanilla. To assess scalability, we compared

TABLE III
PERFORMANCE COMPARISON OF BIGFEAT (BF), ORIGINAL BASE FEATURES (ORG), AND AUTOFEAT (AF) ON DIFFERENT CLASSIFIERS ACROSS DIFFERENT DATASETS

DS	CLF	ORG	AF	SF	BF	BF-m	DS	CLF	ORG	AF	SF	BF	BF-m	DS	CLF	ORG	AF	SF	BF	BF-m
Eeg-eye	LR	**97.25**	97.23	97.23	97.23	97.23	Gina	LR	79.88	79.88	86.00	89.27	**89.92**	Australian	LR	83.87	65.6	**90.77**	89.6	90.41
	AB	97.23	97.23	97.23	97.23	**97.24**		AB	86.45	86.45	86.00	89.27	**89.92**		AB	86.18	85	86.4	85.25	**87.3**
	DT	**97.23**	96.19	95.86	96.31	96.2		DT	85.17	83.35	84	86.94	**87.59**		DT	**85.04**	79.4	77.69	78.63	83.62
	ET	96.25	97.07	**97.32**	96.89	97.11		ET	**94.58**	93.86	94.00	91.91	92.56		ET	78.99	86	88	83.33	**88.9**
	KNN	97.08	97.05	97.08	**97.16**	97.06		KNN	82.57	82.57	83.00	91.70	**92.35**		KNN	**87.8**	55.9	79.69	**87.8**	86.1
	MLP	97.05	96.49	94.26	97.24	**97.27**		MLP	86.15	86.99	88.00	94.79	**95.44**		MLP	67.77	36	77.05	**91.94**	88.1
	RF	96.81	97.42	**97.45**	97.13	97.4		RF	92.72	**93.11**	92.00	92.15	92.80		RF	78.26	86.9	87.8	86.18	**88**
	GB	97.23	**97.26**	97.24	97.25	**97.26**		GB	92.42	92.42	93.00	92.88	**93.53**		GB	63.64	81.7	**85.25**	83.87	84.1
	Avg	97.02	96.99	96.71	97.06	**97.10**		Avg	87.49	87.33	88.25	91.11	**91.76**		Avg	78.94	72.06	84.08	85.83	**87.07**
Madelon	LR	**85.94**	55.95	66.42	68.45	84.53	Kc1	LR	32.56	37.21	25	**38.46**	33.33	GCM	LR	**76.4**	32.5	-	74.74	75.25
	AB	56.24	62.19	73.13	**81.78**	79.59		AB	37.21	34.57	36.36	26.37	**39.08**		AB	**27.57**	18.3	-	22.92	23.6
	DT	62.19	75.4	74.86	77.19	**77.37**		DT	34.57	33.61	38.71	**40.94**	40.65		DT	40.26	45.8	-	49.19	**55.51**
	ET	77.29	70.24	80.07	**89.06**	87.89		ET	36.97	41.76	**43.3**	40.43	42.22		ET	64.28	62.4	-	**69.47**	65.79
	KNN	72.97	72.01	84.17	87.4	**89.25**		KNN	**47.31**	29.55	26.67	36.17	46.43		KNN	**54.72**	27.7	-	50.85	53
	MLP	72.01	56.3	56.3	80.23	**81.99**		MLP	29.55	**41.51**	8.96	36.78	36.14		MLP	0.53	40.3	-	64.41	**77.11**
	RF	30.21	73	81.63	87.6	**87.96**		RF	14.93	41.3	**45.83**	40.86	38		RF	63.77	**71.9**	-	66.88	65.88
	GB	67.63	76.15	84.45	85.94	**87.92**		GB	26.32	36.14	**46.32**	32.56	41.76		GB	63.49	61.3	-	59.42	**72.25**
	Avg	65.56	67.66	75.13	82.21	**84.56**		Avg	32.43	36.96	33.89	36.57	**39.70**		Avg	48.88	45.03	-	57.24	**61.05**
Arcene	LR	68.18	63.39	**82.61**	81.82	81.74	Shuttle	LR	96.73	86.82	56.62	96.77	**97.13**	Numerai28	LR	**57.08**	56.9	56.67	56.22	56.22
	AB	**80.95**	61.75	79.07	71.11	69.41		AB	87.93	87.93	75.82	89.84	**89.9**		AB	**56.9**	55.7	55.78	55.66	55.32
	DT	**68.29**	61.86	59.09	63.64	67.26		DT	99.97	99.97	78.7	**100**	99.99		DT	**55.7**	50.16	50.83	50.29	55.57
	ET	60.87	62.29	**76.19**	68.18	72.73		ET	99.97	**99.99**	84.74	99.96	99.95		ET	50.49	**51.92**	51.52	51.01	51.2
	KNN	71.43	61.1	66.67	**82.93**	80.95		KNN	99.88	98.94	79.09	**99.93**	99.92		KNN	51.76	**51.84**	51.16	51.32	51.04
	MLP	**86.96**	60.26	55.56	70.97	83.26		MLP	**99.99**	97.03	68.94	99.96	99.97		MLP	51.84	3.92	**60.97**	51.55	56.93
	RF	70.97	63.91	69.77	68.18	**73.91**		RF	**99.97**	**99.97**	83.44	99.96	99.96		RF	**65.98**	52	51.38	51.31	64.63
	GB	**74.42**	63.74	71.11	68.18	70.41		GB	**100**	**100**	78.7	99.99	**100**		GB	57.17	56.96	**57.31**	57.08	57.1
	Avg	72.76	62.29	70.01	71.88	**74.96**		Avg	98.06	96.33	75.76	98.3	**98.35**		Avg	55.87	47.43	54.45	53.06	**56.00**
Credit-g	LR	88.28	58.91	**89.04**	**89.04**	88.88	Nomao	LR	**96.98**	92.95	93.46	96.3	96.42	Isolet	LR	87.87	70.6	**94.77**	92.6	92.41
	AB	89.8	87.59	88.44	89.47	**89.86**		AB	93.2	96.16	96.07	96.09	**96.34**		AB	90.18	87	89.4	90.25	**90.3**
	DT	**87.59**	82.67	81.63	80	85.03		DT	96.16	**96.4**	96.23	96.31	96.21		DT	**90.04**	81.4	82.69	83.63	87.62
	ET	84.35	89.04	**92**	88.59	89.8		ET	96.52	**97.98**	97.87	97.83	97.86		ET	83.99	90	92	87.33	**93.9**
	KNN	**92**	70.89	85.52	85.71	91.92		KNN	**97.91**	94.77	94.54	97.13	97		KNN	89.8	57.9	82.69	89.1	**91.8**
	MLP	72.96	69.35	81.12	89.19	**90.74**		MLP	94.96	94.53	94.37	97.29	**97.41**		MLP	69.77	38	80.05	**92.94**	91.1
	RF	78.83	88.44	**89.19**	88.89	88.89		RF	95.37	**97.86**	97.66	97.71	97.75		RF	83.26	88.9	89.8	89.18	**91**
	GB	73.62	**90.54**	88.89	88.28	**90.54**		GB	93.82	96.89	96.9	**96.98**	96.94		GB	66.64	85.7	**89.25**	85.87	86.1
	Avg	83.43	79.68	86.98	87.4	**89.46**		Avg	95.62	95.94	95.89	96.96	**96.99**		Avg	82.69	74.94	87.58	88.86	**90.53**
Blood	LR	**48.57**	24.49	42.11	20.83	21.38	Sonar	LR	**76.72**	0	72.69	55.26	75.06	Texture	LR	**95.25**	93.23	94.23	94.23	95.23
	AB	24.49	51.61	48.57	50	**53.76**		AB	43.56	70.18	68.84	71.87	**72.29**		AB	95.23	92.23	93.23	92.23	**97.24**
	DT	**51.61**	44.8	34.78	47.89	48.44		DT	70.18	77.67	63.19	**78.5**	76.66		DT	92.23	92.19	90.86	91.31	**94.2**
	ET	39.44	42.9	41.18	41.79	**44.78**		ET	77.41	85.57	72.46	85.58	**88.36**		ET	91.25	95.07	**95.32**	92.89	95.11
	KNN	42.25	46.9	**53.13**	52.31	51.48		KNN	**85.67**	53.38	68.34	77.96	76.77		KNN	93.08	95.05	95.08	**95.16**	95.06
	MLP	**54.55**	40	52.78	36.36	40.48		MLP	**79.48**	44.41	69.88	74.76	71.71		MLP	94.05	91.49	89.26	94.24	**96.27**
	RF	28	46.4	43.48	**50.75**	48.48		RF	67.8	83.58	72.83	84.49	**88.51**		RF	93.81	93.42	93.45	94.13	**95.4**
	GB	4.76	43.8	48.48	48.57	**53.56**		GB	74.05	74.83	73.01	76.72	**79.06**		GB	95.23	95.26	93.24	95.25	**97.26**
	Avg	36.71	42.61	**45.56**	43.56	45.30		Avg	71.86	61.20	70.16	75.64	**78.55**		Avg	93.77	93.49	93.08	93.68	**95.72**
Amazon_employee	LR	**97.25**	97.23	97.23	97.23	97.23	Micro-mass	LR	**96.02**	-	-	94.81	94.94	Steel-plates-fault	LR	**76.72**	45.41	42.56	54.26	76.06
	AB	97.23	97.23	97.23	97.23	**97.24**		AB	14.18	-	-	**19.22**	18.15		AB	42.56	70.18	72.18	**73.87**	73.29
	DT	**97.23**	96.19	95.86	96.31	96.2		DT	**86.94**	-	-	86.7	86.48		DT	71.18	79.67	78.67	**80.5**	77.66
	ET	96.25	97.07	**97.32**	96.89	97.11		ET	93.89	-	-	**94.78**	93.76		ET	79.41	84.57	86.57	85.58	**90.36**
	KNN	97.08	97.05	97.08	**97.16**	97.06		KNN	88.74	-	-	**92.55**	91.22		KNN	**84.67**	53.38	52.38	79.96	78.77
	MLP	97.05	96.49	94.26	97.24	**97.27**		MLP	97.28	-	-	96.02	**97.96**		MLP	**79.48**	46.41	45.41	74.76	72.71
	RF	96.81	97.42	**97.45**	97.13	97.4		RF	92.55	-	-	93.76	**94.94**		RF	67.8	85.58	84.58	83.49	**89.51**
	GB	97.23	**97.26**	97.24	97.25	**97.26**		GB	86.46	-	-	**92.05**	91.23		GB	73.05	74.83	75.83	77.72	**80.06**
	Avg	97.02	96.99	96.71	97.06	**97.10**		Avg	82.01	-	-	**83.74**	83.59		Avg	71.86	67.50	67.27	76.27	**79.80**
Christine	LR	**72.98**	-	72.69	67.19	69.5	Phoneme	LR	**76.72**	0	-	55.26	75.06							
	AB	68.34	-	68.84	**71.46**	69.89		AB	43.56	70.18	-	71.87	**72.29**							
	DT	**70.58**	-	63.19	65.31	63.09		DT	70.18	77.67	-	**78.5**	76.66							
	ET	63.38	-	72.46	71.81	**73.03**		ET	77.41	85.57	-	85.58	**88.36**							
	KNN	**74.05**	-	68.34	72.44	73.38		KNN	**85.67**	53.38	-	77.96	76.77							
	MLP	70.59	-	69.88	68.57	**71.48**		MLP	**79.48**	44.41	-	74.76	71.71							
	RF	0	-	**72.83**	71.53	72.37		RF	67.8	83.58	-	84.49	**88.51**							
	GB	74	-	73.01	72.98	**74.54**		GB	74.05	74.83	-	76.72	**79.06**							
	Avg	61.74	-	70.16	70.16	**70.91**		Avg	71.86	61.20	-	75.64	**78.55**							

Note: The configuration that surpasses all other configurations is presented in bold while the configuration with the lowest performance is underlined. Failed runs are presented using hyphens (-).

the default configurations of BigFeat-FE and BigFeat-vanilla to AutoFeat and SAFE by measuring their execution times, as depicted in Fig. 5. The results clearly indicate that BigFeat offers remarkable scalability, boasting an average execution time that is 20 times faster than AutoFeat and 14 times faster than SAFE.

TABLE IV
PERFORMANCE COMPARISON OF THE FEATURE GENERATION AND SELECTION TECHNIQUES OF BIGFEAT-VANILLA/BIGFEAT-FE TO RANDOM FEATURE GENERATION (RFG) AND ANOVA FEATURE SELECTION (AFS) ON DIFFERENT CLASSIFIERS

DS	CLF	RFG	AFS	BF	DS	CLF	RFG	AFS	BF	DS	CLF	RFG	AFS	BF	DS	CLF	RFG	AFS	BF
Eeg-eye	LR	69	70.10	**70**	Gina	LR	87	**90.06**	89.27	Amazon_employee	LR	95.83	95.99	**97.23**	Australian	LR	88.56	87.7	**90.41**
	AB	76	71	**77**		AB	87	88.50	**89**		AB	95.78	96.52	**97.24**		AB	85.67	87.1	**87.3**
	DT	83.09	77	**83**		DT	85	85.47	**86**		DT	**96.38**	94.40	96.2		DT	83.62	**84.1**	83.62
	ET	92.95	87	**92.58**		ET	**93.86**	92	91.91		ET	**97.72**	95.08	97.11		ET	**89.78**	89.1	88.9
	KNN	83.95	80	**84.60**		KNN	**91.46**	91	91		KNN	**97.27**	96.08	97.06		KNN	85.55	85.9	**86.1**
	MLP	68	69.87	**68**		MLP	94	**95.63**	94.79		MLP	94.81	94.49	**97.27**		MLP	**88.79**	88.2	88.1
	RF	91.42	85	**92.04**		RF	91	91.30	**92.15**		RF	94.78	95.29	**97.4**		RF	85.81	85.8	**88**
	GB	84.18	77	**84**		GB	**92.79**	92	92		GB	**97.44**	95.59	97.26		GB	82.64	**85.1**	84.1
	avg	81.27	77	**81.78**		Avg	90	90.88	**91.11**		Avg	96.25	95.43	**97.10**		Avg	86.30	86.62	**87.07**
Madelon	LR	55	**65.35**	63.97	Kc1	LR	**28.77**	23	27.59	Christine	LR	**69.72**	67.57	69.5	GCM	LR	72.44	74.6	**75.25**
	AB	58	73.90	**78.28**		AB	21	27.03	**33**		AB	69.26	67.73	**69.89**		AB	22.76	23.3	**23.6**
	DT	70	**77.91**	73.79		DT	37.97	35	**46**		DT	**63.72**	62.22	63.09		DT	**55.72**	53.3	55.51
	ET	68	**86.89**	85.61		ET	37	38.27	**48**		ET	70.26	71.10	**73.03**		ET	65.47	65.7	**65.79**
	KNN	60	**86.09**	83.05		KNN	28	30.38	**31**		KNN	71.65	70.77	**73.38**		KNN	50.90	52.2	**53**
	MLP	60	**79.30**	74.22		MLP	32.05	20	**34.15**		MLP	69.83	69.58	**71.48**		MLP	76.29	**77.3**	77.11
	RF	67	84.56	**85.31**		RF	35	35.58	**40**		RF	72.10	70.53	**72.37**		RF	**65.92**	63.8	65.88
	GB	70	81.50	**83.81**		GB	21	30.38	**40**		GB	**75.31**	72.86	74.54		GB	69.66	69.9	**72.25**
	avg	64	**79.44**	78.51		Avg	30.49	30	**37.98**		Avg	70.23	69.04	**70.91**		Avg	59.90	60.01	**61.05**
Arcene	LR	83	**86.21**	83.64	Shuttle	LR	91	**94.54**	93.60	Micro-mass	LR	93.01	**95.25**	94.94	Numerai28	LR	56.15	55.7	**56.22**
	AB	**83.02**	76	77.19		AB	96	**96.71**	96.18		AB	**19.12**	15.87	18.15		AB	54.36	54.7	**55.32**
	DT	68.97	56	**76**		DT	**99.96**	99	99		DT	85.63	84.87	**86.48**		DT	**55.68**	54.4	55.57
	ET	77	79.31	**85**		ET	**99.94**	99	99.92		ET	91.74	92.96	**93.76**		ET	48.62	49.5	**51.2**
	KNN	76.67	74	**84.75**		KNN	99.82	99	**99.84**		KNN	88.74	88.36	**91.22**		KNN	49.10	49.1	**51.04**
	MLP	76.19	69	**83**		MLP	99.63	99	**99.66**		MLP	**98.27**	97.44	97.96		MLP	55.38	56.1	**56.93**
	RF	77.19	75	**78**		RF	**99.94**	99	99.92		RF	**95.78**	95.55	94.94		RF	64.12	61.7	**64.63**
	GB	60	77.42	**80**		GB	**99.96**	99	99		GB	88.87	88.59	**91.23**		GB	55.48	**57.1**	57.1
	avg	75.41	74	**81.29**		Avg	98	**98.69**	98.63		Avg	82.64	82.36	**83.58**		Avg	54.86	54.79	**56.00**
Credit-g	LR	80	**84.48**	81.03	Nomao	LR	**91**	90.92	**91**	Phoneme	LR	72.23	72.61	**75.06**	Isolet	LR	89.42	**92.9**	92.41
	AB	81	**85.94**	81.97		AB	**91**	90.74	**91**		AB	**72.75**	72.17	72.29		AB	**90.66**	88.3	90.3
	DT	78.02	**83.92**	76		DT	90.42	89	**90.61**		DT	74.61	75.78	**76.66**		DT	**87.84**	86.4	87.62
	ET	82.93	**86.29**	82		ET	**94.90**	94	94.73		ET	86.39	87.06	**88.36**		ET	**94.66**	93.7	93.9
	KNN	81.95	**85.38**	82.38		KNN	**92.51**	92	**92.51**		KNN	76.96	**77.47**	76.77		KNN	90.71	89.9	**91.8**
	MLP	81	**84.77**	81.36		MLP	**93.13**	91	93		MLP	69.90	71.46	**71.71**		MLP	90.11	**91.4**	91.1
	RF	82.63	**86.07**	83.56		RF	**94.84**	93	94.56		RF	87.57	87.44	**88.51**		RF	88.35	**91.5**	91
	GB	83.23	**85.12**	83.97		GB	92.87	92	**93**		GB	76.84	**79.90**	79.06		GB	**86.38**	85.2	86.1
	avg	81.50	**85.25**	81.61		Avg	92.67	91	**92.730**		Avg	77.16	77.99	**78.55**		Avg	89.77	89.92	**90.53**
Blood	LR	85.11	84	**85**	Sonar	LR	75	78.38	**84.21**	Steel-plates-fault	LR	74.40	73.77	**76.06**	Texture	LR	95.16	95.0	**95.23**
	AB	82	**84.60**	83.33		AB	81.58	76	**84**		AB	71.11	72.97	**73.29**		AB	95.73	96.1	**97.24**
	DT	77	**83.71**	79.79		DT	74.36	70	**77**		DT	**78.43**	75.85	77.66		DT	**94.78**	92.6	94.2
	ET	81	**84.16**	83.29		ET	**91.14**	78	82.50		ET	88.62	87.47	**90.36**		ET	**95.22**	93.0	95.11
	KNN	83	81.93	**83.38**		KNN	82.93	76	**85**		KNN	**79.64**	76.44	78.77		KNN	93.71	**96.0**	95.06
	MLP	84.13	**84.96**	84.54		MLP	81.08	80	**87**		MLP	71.26	71.87	**72.71**		MLP	94.41	96.2	**96.27**
	RF	82	82.44	**83**		RF	82.05	76	**82.50**		RF	**89.96**	89.33	89.51		RF	94.36	92.7	**95.4**
	GB	83	84.29	**84**		GB	78.95	78.95	**84**		GB	77.72	**80.13**	80.06		GB	**97.34**	96.0	97.26
	avg	82	**83.85**	83.46		Avg	80.89	77	**83.67**		Avg	78.89	78.48	**79.80**		Avg	95.09	94.72	**95.72**

Note: The configuration that surpasses all other configurations is presented in bold while the configuration with the lowest performance is underlined.

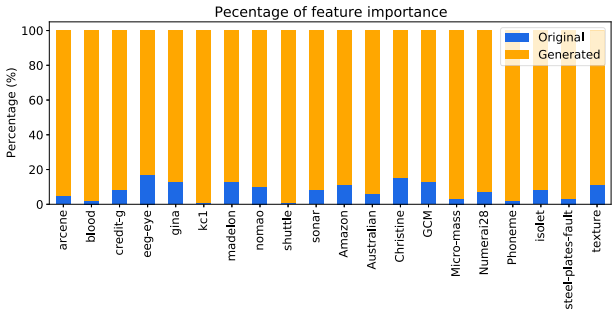

Fig. 3. Contribution of generated and original feature set.

G. Meta-Learning Evaluation in the BigFeat-FE

To comprehensively assess the significance of the meta-learning phase within BigFeat-FE and to evaluate the effectiveness of the meta-features employed for data representation, we conducted a thorough analysis. We compared the performance of BigFeat-FE based on seven distinct sets of features: 1) the comprehensive feature set utilized in BigFeat-FE, comprising statistical, information-theoretic, landmarkers, and model-based features (referred to as BF-StatLandMod); 2) a feature set exclusively comprised of statistical features (referred to as BF-Stat); 3) a feature set consisting solely of landmarkers metafeatures (referred to as BF-Land); 4) a feature set comprising only model-based metafeatures (referred to as BF-Mod); 5) a feature set combining landmarkers and model-based metafeatures (referred to as BF-LandMod); 6) a feature set comprising statistical, information-theoretic, and model-based metafeatures (referred to as BF-StatMod); and 7) a feature set including statistical, information-theoretic, and landmarkers metafeatures (referred to as BF-StatLand). This comprehensive examination allowed us to perceive the individual and collective contributions of these feature categories to the performance of BigFeat-FE, providing valuable insights into their

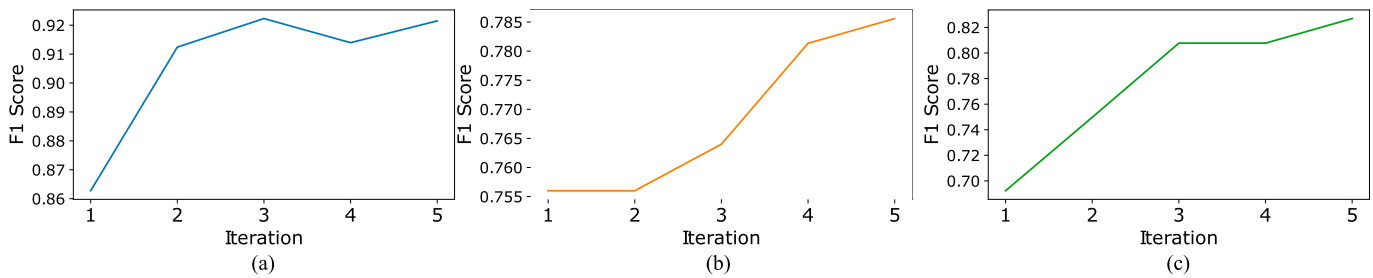

Fig. 4. Performance improvements over iterations. (a) Eeg-eye. (b) Credit-g. (c) Sonar.

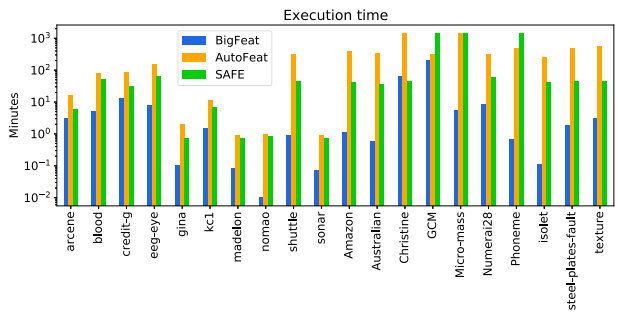

Fig. 5. Execution time of BigFeat-FE versus AutoFeat and SAFE. Y-axis is log-scaled for presentation purposes.

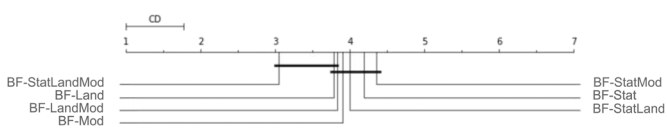

Fig. 6. Nemenyi test ($\alpha = 0.05$) of the performance of meta-learning-based techniques for warm starting including BF-StatLandMod, BF-Land, BF-Stat, BF-StatLand, BF-StatMod, BF-LandMod, and BF-Mod.

respective significance within the meta-learning phase. The results of this analysis revealed that BF-StatLandMod consistently enhances the performance of BigFeat-FE across 19 out of 20 datasets with average difference of 1.83% (See Table III). This observation underscores the pivotal role of BF-StatLandMod features in augmenting the effectiveness of BigFeat-FE. Furthermore, the Nemenyi signed-rank test confirmed the statistically significant superiority of BigFeat-FE's performance based on BF-StatLandMod over BF-Stat, BF-Mod, BF-StatMod, and StatLand, as illustrated in Fig. 6. We conducted the Nemenyi test to assess the significance of performance differences between BigFeat-FE, BigFeat-vanilla, SAFE, AutoFeat, and ORG. The results of the Nemenyi signed-rank test confirm the statistically significant superiority of BigFeat-FE's performance, as illustrated in Fig. 7.

H. Interpretability

As interpretability is one of the main features of BigFeat, it is critical for each generated feature to be viewed as an aggregation of a set of mathematical operations over the base features. Therefore, BigFeat easily map the generated features

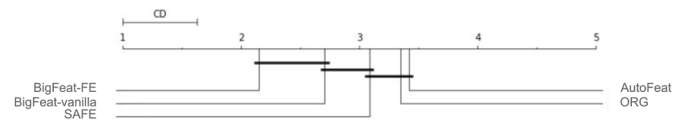

Fig. 7. Nemenyi test ($\alpha = 0.05$) of the performance of BigFeat-FE, BigFeat-vanilla, SAFE AutoFeat, and ORG.

TABLE V
CLASSIFICATION PERFORMANCE BIGFEAT AUTOML AGAINST AUTOSKLEARN AND TPOT

Dataset	AutoSKLearn	TPOT	BigFeat-model2
Arcene	80.00	**90.00**	**90.00**
Blood	79.13	78.07	**80.40**
Credit-g	**76.40**	73.60	75.35
Eeg-eye	92.31	**97.52**	92.47
Gina	92.65	94.68	**94.99**
Kc1	84.47	86.74	**87.32**
Madelon	75.85	84.92	**88.81**
Nomao	96.79	96.24	**96.87**
Shuttle	**99.98**	99.96	99.94
Sonar	75.00	**84.61**	84.29
Amazon_employee	**99.53**	97.88	98.94
Australian	88.07	88.15	**89.92**
Christine	**72.93**	72.35	72.56
GCM	63.19	**63.63**	62.69
Micro-mass	90.00	90.59	**91.71**
Numerai28	57.01	**58.61**	58.57
Phoneme	78.57	78.57	**81.30**
Isolet	93.09	90.69	**93.31**
Steel-plates-fault	82.37	80.87	**82.73**
Texture	97.21	97.14	**97.38**
Avg	83.73	85.24	**85.98**

Note: The configuration that surpasses all other configurations is presented in bold while the configuration with the lowest performance is underlined.

to the original features using a simple mathematical formula. For example, the Euclidean distance feature in Fig. 2(a) is expressed as: $\mathbb{X}^i = \sqrt{(X^{34} - X^{76})^2 - (X^{53} - X^{44})^2}$ and the price per meter feature in Fig. 2(b) is expressed as $\mathbb{X}^i = X^{64} \div (X^{27} \times X^{47})$. These simple expression could be easily explained and mapped to the original features, especially those generated from shallow computational trees.

I. Limitations

BigFeat-FE is a FE approach that achieves linear time and space complexity by optimizing the feature generation and selection phases. However, it may not always produce optimal features as the use of IG as a feature importance criterion

TABLE VI
OVERVIEW OF THE USED META-FEATURES

Name	Formula	Rationale	Additional Variants
Nr instances	n	Speed, scalability	n, $\log(n)$, $\frac{p}{n}$, X, $(n-X)$
Nr features	p	Curse of dimensionality	p, $\log(p)$, Nr|min|μ|$\sigma(\pi_i)$
Nr classes	c	Complexity, imbalance	c, min|max|$\sigma(\frac{c}{n})$
Nr missing values	m	Imputation effects	m, $\frac{m}{n}$
Skewness	$\frac{E(X-\mu_X)^3}{\sigma_X^3}$	Feature normality	Min, max, μ, σ, q_1, q_3
Kurtosis	$\frac{E(X-\mu_X)^4}{\sigma_X^4}$	Feature normality	Min, max, μ, σ, q_1, q_3
Correlation	$\rho_{X_1X_2}$	Feature interdependence	Min, max, μ, σ
Class probability	$P(\mathtt{C})$	Class distribution	Min, max, μ,σ
Class entropy	$H(\mathtt{C})$	Class imbalance	$H(\mathtt{C})/\mu(MI(\mathtt{C},\mathtt{X}))$
Norm. entropy	$\frac{H(\mathtt{X})}{\log_2 n}$	Feature informativeness	Min, max, μ, σ
Mutual inform.	$MI(\mathtt{C},\mathtt{X})$	Feature importance	Min, max, μ, σ
Equiv. nr. feats	$\frac{H(C)}{MI(C,X)}$	Intrinsic dimensionality	
Noise-signal ratio	$\frac{H(X)-MI(C,X)}{MI(C,X)}$	Noisiness of data	
Nr nodes, leaves	$\|\eta\|$, $\|\psi\|$	Concept complexity	μ|σ (tree depth), μ|σ[Nodes per (attr, inst, level)]
Branch length		Concept complexity	μ, σ
Leaves corroboration		Concept complexity	μ, σ
Leaves homogeneity		Class complexity	μ, σ
Nodes per feature	$\|\eta_X\|$	Feature importance	Min, max, μ, σ
Leaves per class	$\frac{\|\psi_c\|}{\|\psi\|}$	Class complexity	μ, σ
Nodes per level		Concept complexity	μ, σ
Leaves agreement	$\frac{n_{\psi_i}}{n}$	Class separability	Min, max, μ, σ
Landmarker (1NN)	$P(\theta_{\text{1NN}}, t_j)$	Data sparsity	μ|σ, μ|σ (eliteNN)
Landmarker (Tree)	$P(\theta_{\text{Tree}}, t_j)$	Data separability	μ|σ (bestNode, randomNode, worestNode)
Landmarker (Lin)	$P(\theta_{\text{Lin}}, t_j)$	Linear separability	μ|σ (Lin.disciminant)
Landmarker (NB)	$P(\theta_{\text{NB}}, t_j)$	Feature independence	μ, σ

Note: Groups from top to bottom: statistical and information-theoretic, model-based, and landmarkers. Continuous features X and target Y have mean μ_X, stdev σ_X, variance σ_X^2. Categorical features $\mathtt{X}$ and class $\mathtt{C}$ have categorical values π_i, conditional probabilities $\pi_{i|j}$, joint probabilities $\pi_{i,j}$, marginal probabilities $\pi_{i+} = \sum_j \pi_{ij}$, entropy $H(\mathtt{X}) = -\sum_i \pi_{i+}\log_2(\pi_{i+})$ [56].

in the generation phase may not accurately represent feature importance for every machine learning model. To overcome this limitation and ensure interpretability, BigFeat-AutoML has constrained its machine learning search space to three decision tree-based models and a single linear model. In the feature selection phase, BigFeat is designed to check the similarity between parent features and their offspring, which are features generated from the parent features. This approach significantly reduces the number of comparisons required, as compared to comparing each generated/original pair of features. However, for datasets with a large number of similar features, this approach may not be optimal and could lead to redundancy in the selected features.

VII. EVALUATING BIGFEAT-AUTOML

To comprehensively assess the effectiveness of BigFeat-AutoML as a complete end-to-end AutoML solution, we conducted a comparative analysis against prominent AutoML tools, specifically TPOT and AutoSKLearn. The performance of BigFeat-AutoML, along with the baseline methods, is summarized in Table V. Our results indicate that BigFeat-AutoML outperforms TPOT on 15 datasets and AutoSKLearn on 16 datasets. The average performance difference, as detailed in Table V, highlights BigFeat-AutoML's significant edge over AutoSKLearn and TPOT, with an average improvement of 2.25% and 0.74%, respectively.

VIII. CONCLUSION AND FUTURE WORK

This article introduces BigFeat, a framework designed to automate critical stages of the machine learning pipeline, encompassing FE, model selection, and hyperparameter tuning. BigFeat stands out due to its focus on scalability, interpretability, model-agnosticism, and pipeline-enablement. In our extensive experiments, we have empirically demonstrated the exceptional performance of BigFeat-FE, consistently surpassing state-of-the-art automated FE frameworks AutoFeat and SAFE. Across a diverse array of datasets, BigFeat-FE achieved an impressive average performance improvement of 8.65% compared to AutoFeat and 4.71% compared to SAFE, highlighting its efficacy. Furthermore, when evaluated as BigFeat-AutoML, our framework exhibited even more substantial performance gains compared to TPOT and Autosklearn, achieving an average improvement of 0.74% compared to TPOT and 2.25% compared to Autosklearn, respectively. In our future endeavors, we plan to expand the framework's capabilities to include data cleaning phase and support datatype-based transformation, especially for image and text data.

APPENDIX

A. Default Configuration

The parameter selection for BigFeat is meticulously designed to align with the average requirements across a broad spectrum

of datasets. The determination of the optimal values for parameters such as K, α, and the number of iterations is governed by a balancing act between the generation of increasingly complex features and the efficient utilization of computational resources. In general, the introduction of more sophisticated features tends to enhance the predictive performance of our tree-based model. However, it is important to note that these enhancements do not universally translate to improved performance across different models. Consequently, we have incrementally adjusted these parameters with a deliberate focus on avoiding overfitting within our decision tree model. In the case of η, our approach involved the exclusion of features exhibiting high linear correlation, thereby addressing a similar tradeoff between feature uniqueness and model information loss.

B. Metafeatures of the Offline Phase

Table VI presents the meta-features employed during the offline phase, showing their mathematical formulations and underlying rationale.

Acknowledgment

The authors would like to thank the student Kayahan Kaya for his involvement in some of the experiments of this work.

References

[1] A. Y. Zomaya and S. Sakr, *Handbook of Big Data Technologies*. Cham, Switzerland: Springer-Verlag, 2017.

[2] J. Bastos, "Credit scoring with boosted decision trees," Munich Personal RePEc Archive (MPRA) Paper No. 8034. University Library of Munich, Germany. 2007. [Online]. Available: https://mpra.ub.uni-muenchen.de/8034/

[3] J. O. Awoyemi, A. O. Adetunmbi, and S. A. Oluwadare, "Credit card fraud detection using machine learning techniques: A comparative analysis," in *Proc. Int. Conf. Comput. Netw. Inform. (ICCNI)*. Piscataway, NJ, USA: IEEE Press, 2017, pp. 1–9.

[4] L. Ma and B. Sun, "Machine learning and AI in marketing–connecting computing power to human insights," *Int. J. Res. Marketing*, vol. 37, no. 3, pp. 481–504, 2020.

[5] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1798–1828, Aug. 2013.

[6] R. Elshawi and S. Sakr, "Automated machine learning: Techniques and frameworks," in *Proc. Big Data Manage. Analytics: 9th Eur. Summer School (eBISS)*, Berlin, Germany. Springer-Verlag, 2020, pp. 40–69.

[7] C. Thornton, F. Hutter, H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms," in *Proc. 19th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2012, pp. 847–855.

[8] M. Feurer, K. Eggensperger, S. Falkner, M. Lindauer, and F. Hutter, "Auto-sklearn 2.0: The next generation," 2020, *arXiv:2007.04074*.

[9] R. S. Olson and J. H. Moore, "TPOT: A tree-based pipeline optimization tool for automating machine learning," in *Proc. Workshop Autom. Mach. Learn.*, 2016, pp. 66–74.

[10] E. LeDell and S. Poirier, "H2O AutoML: Scalable automatic machine learning," in *Proc. 7th ICML Workshop Automated Mach. Learn. (AutoML)*, Jul. 2020. [Online]. Available: https://www.automl.org/wp-content/uploads/2020/07/AutoML_2020_paper_61.pdf

[11] D. Wang et al.,"AutoAI: Automating the end-to-end AI lifecycle with humans-in-the-loop," in *Proc. 25th Int. Conf. Intell. User Interfaces Companion*, 2020, pp. 77–78.

[12] H. T. Lam et al., "Automated data science for relational data," in *Proc. IEEE 37th Int. Conf. Data Eng. (ICDE),* Piscataway, NJ, USA: IEEE Press, 2021, pp. 2689–2692.

[13] H. T. Lam, T. N. Minh, M. Sinn, B. Buesser, and M. Wistuba, "Neural feature learning from relational database," 2018, *arXiv:1801.05372*.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[15] F.-Y. Wang et al., "Where does AlphaGo go: From church-turing thesis to AlphaGo thesis and beyond," *IEEE/CAA J. Automatica Sinica*, vol. 3, no. 2, pp. 113–120, Apr. 2016.

[16] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, pp. 279–292, May 1992.

[17] X. Chen et al., "Neural feature search: A neural architecture for automated feature engineering," in *Proc. IEEE Int. Conf. Data Mining (ICDM),* Piscataway, NJ, USA: IEEE Press, 2019, pp. 71–80.

[18] F. Horn, R. Pack, and M. Rieger, "The autofeat python library for automated feature engineering and selection," in *Proc. Mach. Learn. Knowl. Discovery Databases: Int. Workshops (ECML PKDD)*, Würzburg, Germany. Würzburg, Germany: Springer-Verlag, 2020, pp. 111–120.

[19] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis: Towards automating data science endeavors," in *Proc. IEEE Int. Conf. Data Sci. Adv. Analytics (DSAA)*, Paris, France. Piscataway, NJ, USA: IEEE Press, 2015, pp. 1–10.

[20] G. Zhu, Z. Xu, C. Yuan, and Y. Huang, "DIFER: differentiable automated feature engineering," in *Proc. Int. Conf. Automated Mach. Learn.,* PMLR, 2022, pp. 17–1.

[21] H. Eldeeb, S. Amashukeli, and R. ElShawi, "BigFeat: Scalable and interpretable automated feature engineering framework," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 515–524.

[22] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, 2015, Art. no. 436.

[23] R. El Shawi, Y. Sherif, and S. Sakr, "Towards automated concept-based decision treeExplanations for CNNs," in *Proc. 24th Int. Conf. Extending Database Technol. (EDBT)*, 2021, pp. 379–384.

[24] G. Montavon, W. Samek, and K.-R. Müller, "Methods for interpreting and understanding deep neural networks," *Digit. Signal Process.*, vol. 73, pp. 1–15, Feb. 2018.

[25] R. Elshawi, M. H. Al-Mallah, and S. Sakr, "On the interpretability of machine learning-based model for predicting hypertension," *BMC Med. Inform. Decis. Making*, vol. 19, no. 1, pp. 1–32, 2019.

[26] F. Horn, R. Pack, and M. Rieger, "The autofeat Python library for automated feature engineering and selection," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases,* Springer-Verlag, 2019, pp. 111–120.

[27] S. Piramuthu and R. T. Sikora, "Iterative feature construction for improving inductive learning algorithms," *Expert Syst. Appl.*, vol. 36, no. 2, pp. 3401–3406, 2009.

[28] W. Fan et al., "Generalized and heuristic-free feature construction for improved accuracy," in *Proc. SIAM Int. Conf. Data Mining,* Philadelphia, PA, USA: SIAM, 2010, pp. 629–640.

[29] A. Kaul, S. Maheshwary, and V. Pudi, "Autolearn—Automated feature generation and selection," in *Proc. IEEE Int. Conf. Data Mining (ICDM),* Piscataway, NJ, USA: IEEE Press, 2017, pp. 217–226.

[30] U. Khurana, D. Turaga, H. Samulowitz, and S. Parthasrathy, "Cognito: Automated feature engineering for supervised learning," in *Proc. IEEE 16th Int. Conf. Data Mining Workshops (ICDMW),* Piscataway, NJ, USA: IEEE Press, 2016, pp. 1304–1307.

[31] Q. Shi, Y.-L. Zhang, L. Li, X. Yang, M. Li, and J. Zhou, "SAFE: Scalable automatic feature engineering framework for industrial tasks," in *Proc. IEEE 36th Int. Conf. Data Eng. (ICDE),* Piscataway, NJ, USA: IEEE Press, 2020, pp. 1645–1656.

[32] F. Nargesian, H. Samulowitz, U. Khurana, E. B. Khalil, and D. S. Turaga, "Learning feature engineering for classification." in *Proc. 26th Int. Joint Conf. Artif. Intell.,* 2017, pp. 2529–2535.

[33] U. Khurana, H. Samulowitz, and D. Turaga, "Feature engineering for predictive modeling using reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 3407–3414.

[34] G. Lu et al., "Catch: Collaborative feature set search for automated feature engineering," in *Proc. ACM Web Conf.*, 2023, pp. 1886–1896.

[35] G. Katz, E. C. R. Shin, and D. Song, "ExploreKit: Automatic feature generation and selection," in *Proc. IEEE 16th Int. Conf. Data Mining (ICDM),* Piscataway, NJ, USA: IEEE Press, 2016, pp. 979–984.

[36] N. Meinshausen and P. Bühlmann, "Stability selection," *J. Roy. Statist. Soc.: Ser. B (Statist. Methodol.)*, vol. 72, no. 4, pp. 417–473, 2010.

[37] T. Lange, M. L. Braun, V. Roth, and J. M. Buhmann, "Stability-based model selection," in *Proc. Adv. Neural Inf. Process. Syst. 15 (NIPS)*, vol. 15, 2002, pp. 633–642.

[38] J. Lee Rodgers and W. A. Nicewander, "Thirteen ways to look at the correlation coefficient," *Amer. Statist.*, vol. 42, no. 1, pp. 59–66, 1988.

[39] M. Feurer, J. T. Springenberg, and F. Hutter, "Initializing Bayesian hyperparameter optimization via meta-learning," in *Proc. 29th AAAI Conf. Artif. Intell.*, 2015, pp. 1128–1135.

[40] M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2962–2970.

[41] R. ElShawi and S. Sakr, "TPE-AutoClust: A tree-based pipline ensemble framework for automated clustering," in *Proc. IEEE Int. Conf. Data Mining Workshops (ICDMW),* Piscataway, NJ, USA: IEEE Press, 2022, pp. 1144–1153.

[42] P. Brazdil, C. G. Carrier, C. Soares, and R. Vilalta, *Metalearning: Applications to Data Mining*. Berlin, Germany: Springer Science & Business Media, 2008.

[43] D. W. Aha, "Generalizing from case studies: A case study," in *Machine Learning Proceedings.* Aberdeen, United Kingdom: Elsevier, 1992, pp. 1–10.

[44] J. Gama and P. Brazdil, "Characterization of classification algorithms," in *Proc. Portuguese Conf. Artif. Intell.,* Madeira Island, Portugal: Springer-Verlag, 1995, pp. 189–200.

[45] H. Bensusan, C. G. Giraud-Carrier, and C. J. Kennedy, "A higher-order approach to meta-learning," *ILP Work Progress Rep.*, vol. 35, 2000, Art. no. 44.

[46] Y. Pavel and B. C. Soares, "Decision tree-based data characterization for meta-learning," in *Proc. Integr. Collaboration Aspects Data Mining, Decis. Support Meta-Learn. (IDDM)*, vol. 111, 2002.

[47] B. Pfahringer, H. Bensusan, and C. G. Giraud-Carrier, "Meta-learning by landmarking various learning algorithms." in *Proc. 17th Int. Conf. Mach. Learn. (ICML),*, 2000, pp. 743–750.

[48] H. Bensusan and C. Giraud-Carrier, "Discovering task neighbourhoods through landmark learning performances," in *Eur. Conf. Princ. Data Mining Knowl. Discovery*, Lyon, France: Springer-Verlag, 2000, pp. 325–330.

[49] L. Buitinck et al., "API design for machine learning software: Experiences from the scikit-learn project," in *Proc. ECML PKDD Workshop: Lang. Data Mining Mach. Learn.*, 2013, pp. 108–122.

[50] B. Bischl et al., "OpenML benchmarking suites and the OpenML100," 2017, *arXiv:1708.03731*.

[51] B. Bischl et al., "OpenML benchmarking suites," 2019, *arXiv: 1708.03731v2*.

[52] F. Pedregosa et al., "Scikit-learn: machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.

[53] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *Proc. Int. Conf. Learn. Intell. Optim.*, 2011, pp. 507–523.

[54] R. S. Olson, N. Bartley, R. J. Urbanowicz, and J. H. Moore, "Evaluation of a tree-based pipeline optimization tool for automating data science," in *Proc. Genetic Evol. Comput. Conf.,* New York, NY, USA: ACM, 2016, pp. 485–492.

[55] H. Eldeeb, M. Maher, R. Elshawi, and S. Sakr, "AutoMLBench: A comprehensive experimental evaluation of automated machine learning frameworks," *Expert Syst. Appl.*, vol. 243, 2024, Art. no. 122877.

[56] J. Vanschoren, "Meta-learning: A survey," 2018, *arXiv:1810.03548*.